

내가 기존에 작성한 '지옥의 토폴 벌금 관리기' 절차지향 코드를 과제 조건(무한루프 메뉴, 함수 분리, 매개변수/반환값, 전역/지역 변수 구분)에 맞춰 리팩토링하려고 해. 원본 계산식이나 로직을 훼손하지 않으면서 코드를 분리할 수 있는 전체적인 구조 설계(배대)를 먼저 제안해 줄래?

작성하신 '지옥의 토폴 벌금 관리기' 코드는 이미 논리적인 흐름이 매우 탄탄합니다. 이 절차지향 코드를 과제 조건에 맞춰 모듈화(함수화)한다면, 각 기능의 역할에 따라 크게 4개의 함수와 1개의 메인 루프로 배대를 나누는 것을 제안합니다.

다음과 같은 구조로 분리하면 원본 로직을 100% 유지하면서 과제 조건을 모두 만족할 수 있습니다.

📌 모듈화 구조 설계 청사진

1. 전역 변수 설정 구역 (Global Variables)

- 프로그램 전체에서 상태를 공유해야 하는 변수들(방장 이름, 조 이름, 기본 벌금, 전체 벌금 리스트 `fees` 등)은 최상단에 배치하여 관리합니다.

2. `main()` 함수 (무한 루프 및 제어)

- `while True` 를 사용하여 1~4번 메뉴(입력, 조회, 분석, 종료)를 화면에 출력합니다.
- 사용자의 입력에 따라 아래의 세부 기능 함수들을 '목차처럼' 호출하는 통제 센터 역할을 합니다.

3. `input_member_status()` 함수 (데이터 입력 담당)

- 메뉴 1번을 선택했을 때 호출됩니다.
- 기존 코드에 있던 `for` 문을 가져와서, 조원들의 지각/무단결석 상태를 입력받는 역할을 전담합니다.

4. `calculate_fee(status, mins, name)` 함수 (핵심 연산 담당)

- 입력 함수로부터 상태와 지각 시간 등을 매개변수(인자)로 전달받습니다.
- 벌금 계산 로직을 수행하며, 이때 계산에만 임시로 쓰이는 변수들(`calc_fee` 등)은 지역 변수로 처리하여 메모리 스코프를 분리합니다.
- 계산이 완료된 최종 벌금액을 `return` 문으로 반환합니다.

5. `analyze_data(fees_list)` 함수 (데이터 분석 담당)

- 메뉴 3번을 선택했을 때 호출됩니다.
- 전체 벌금 리스트를 매개변수로 전달받아, 가장 비싼 벌금액을 찾아낸 뒤 그 값을 `return` 합니다.

이렇게 배대를 잡으면 교수님이 요구하신 5가지 조건(무한루프, 함수 최소 3개 분리, 매개변수 전달, 반환값 활용, 전역/지역 변수 구분)을 아주 깔끔하게 충족할 수 있습니다.

전체적인 구조는 이렇게 짜면 될 것 같은데, 혹시 머릿속으로 기존 코드를 이 구조로 옮긴다고 시뮬레이션해 보셨을 때 논리적으로 충돌하거나 구현이 까다로울 것 같은 부분은 없으신가요?

👍 👎 🔄 🗑️ ...

코드를 짜다 보니 문제가 하나 생겼어. 기존 코드에서는 무단결석(상태 2)을 입력받으면 break를 써서 그 즉시 조건 반복문(for문)을 강제 종료하고 정산을 파토냈거든? 그런데 계산 로직을 별도의 함수로 분리해 버리면, 함수 내부에서 밖의 for문을 멈출 수가 없잖아. 이 문제를 어떻게 해결하면 좋을지 아이디어도 같이 논의해 줘.

정말 날카롭고 훌륭한 질문입니다! 코드를 단순히 잘라 붙이는 수준을 넘어서, 함수의 스코프(Scope)와 제어 흐름(Control Flow)이 어떻게 동작하는지 정확히 꿰뚫어 보셨네요.

말씀하신 대로 `break` 는 자신이 속한 가장 가까운 반복문만 멈출 수 있습니다. 계산 로직을 `calculate_fee` 라는 함수로 분리해 버리면, 그 안에서 아무리 `break` 를 외쳐도 바깥쪽 `input_member_status` 함수에서 돌고 있는 `for` 문에는 닿지 않죠.

이 문제를 해결하기 위해 파이썬의 아주 유용한 기능인 '다중 반환(Multiple Return)' 구조를 활용하는 것을 제안합니다.

💡 해결 아이디어: 파토 여부(신호등) 함께 반환하기

계산 함수가 단순히 '별금액(숫자)' 하나만 던져주는 게 아니라, "이번에 텀플이 파토 났으니 반복문을 멈춰야 해!"라는 신호(True/False)를 같이 던져주도록 설계하는 것입니다.

구체적인 로직은 다음과 같이 구현할 수 있습니다.

1. `calculate_fee` 함수 (값을 던져주는 쪽)

무단결석(상태 2)일 때는 별금액과 함께 `True` (파토남)를 반환하고, 일반 지각/출석일 때는 별금액과 함께 `False` (파토 안 남)를 반환합니다.

Python

```
def calculate_fee(status, mins, name):
    # (조건 변수 를 선택)

    if status == 2:
        print(f"무단결석 적발!! {name}님의 결석으로 오는 월급은 파토났습니다!!")
        # 최대 별급과 함께 '파토 여부(True)' 를 반환!
        return max_fee, True

    elif status == 1:
        # 결석 출석이나 일반 지각일 경우 별급과 함께 'False' 반환!
        # ... (지각 계산 로직) ...
        return fee, False
```

2. `input_member_status` 함수 (값을 받아서 제어하는 쪽)

메인 반복문이 도는 곳에서는 함수가 던져준 두 개의 값을 각각 `fee` 와 `is_break` 라는 변수로 나란히 받아 냅니다. 그리고 `is_break` 가 `True` 일 때 밖에서 `break` 를 걸어줍니다.

Python

```
def input_member_status():
    # ... (for문 시작) ...
    for name in names:
        status = int(input("상태를 입력하세요:"))

        # 함수가 던져준 두 개의 값을 나란히 받기
        fee, is_break = calculate_fee(status, mins, name)

        fees.append(fee)
        total += fee

        # 만약 함수에서 True(파토남)를 던졌다면 여기서 반복문 강제 종료!
        if is_break == True:
            break
```

이렇게 구성하면 기존의 "무단결석 시 정산 즉시 중단"이라는 핵심 로직을 100% 보존하면서도, 코드의 모듈 화를 완벽하게 이뤄낼 수 있습니다.

어떠신가요? 이 아이디어를 적용해서 코드를 수정해 보시면 교수님이 요구하신 '반환값(return)의 적극적 활용' 조건까지 아주 훌륭하게 만족하는 결과물이 나올 것 같습니다!